

DAFTAR PERTANYAAN PENGUJI & KESALAHAN KONSEP

Proposal: Analisis Sentimen Pengguna Platform X Terhadap Game Roblox Pada Komunitas Roblox Indonesia Menggunakan IndoBERTweet Dan BERTopic

Nadiatul Mawaddah - F1E122072

Catatan untuk penguji: Dokumen ini disusun sebagai panduan pertanyaan sidang dan daftar temuan konseptual dalam proposal. Pertanyaan disusun berurutan sesuai halaman dan bagian dokumen.

Setiap pertanyaan dilengkapi dengan jawaban ideal berdasarkan best practices yang terverifikasi dari literatur dan web search.

A. PERTANYAAN UNTUK SIDANG (BERURUTAN PER BAGIAN)

BAB I - PENDAHULUAN

Pertanyaan 1 — Definisi "Komunitas" di Platform X (hal. 1)

Mahasiswa menyebutkan bahwa data diambil dari "Komunitas Roblox Indonesia" di Platform X.

"Jelaskan apa yang dimaksud dengan 'Komunitas Roblox Indonesia' di Platform X? Apakah ini komunitas resmi atau buatan pengguna?"

Jawaban Ideal:

- Fitur **Communities** di X adalah fitur grup tertutup/semi-tertutup yang diperkenalkan Twitter pada 2021
 - Komunitas memiliki moderator dan aturan sendiri
 - Data dari Communities berbeda dengan data dari hashtag umum atau timeline publik
 - Mahasiswa harus menjelaskan apakah data diambil dari:
 - ☐ Komunitas resmi (perlu membership)
 - ☐ Hashtag publik (#RobloxIndonesia)
 - ☐ Akun-akun yang membahas Roblox
 - **Implikasi metodologis:** Karakteristik bahasa dan topik bisa berbeda tergantung sumber
-

Pertanyaan 2 — Justifikasi Pemilihan Roblox sebagai Objek Penelitian (hal. 1-2)

"Berapa jumlah pengguna Roblox di Indonesia? Mengapa Roblox dipilih dibandingkan game lain?"

Jawaban Ideal: Berdasarkan data terverifikasi dari [Backlinko](#) dan [Statista](#):

- Q3 2024: 88,9 juta DAU global
- Q3 2025: 151,5 juta DAU global (naik 70% YoY)
- Asia-Pacific memiliki 46,3 juta DAU (terbesar)
- Indonesia termasuk dalam region Asia-Pacific

Justifikasi yang seharusnya:

- Roblox memiliki basis pengguna muda yang besar
- Platform user-generated content yang unik
- Kombinasi gaming + social platform + kreasi digital
- Belum banyak penelitian sentimen Roblox berbahasa Indonesia

Pertanyaan 3 — Kecukupan Data 7 Hari (hal. 1, 29)

"Apakah data 7 hari (6-13 Desember 2025) cukup representatif? Bagaimana menghindari bias temporal?"

Jawaban Ideal: Berdasarkan best practices dari [PMC](#) dan literatur Twitter sentiment analysis:

Durasi	Kegunaan	Risiko
1 minggu	Analisis event spesifik	Bias tinggi, tidak representatif
1-3 bulan	Analisis tren umum	Risiko sedang
6+ bulan	Analisis longitudinal	Risiko rendah, data lebih representatif

Rekomendasi:

- Minimal **4-8 minggu** untuk menangkap variasi temporal
- Jika tetap 7 hari, harus:
 1. Jelaskan alasan pemilihan periode tersebut
 2. Verifikasi tidak ada event/kontroversi besar pada minggu itu
 3. Sebutkan sebagai **keterbatasan penelitian**

Pertanyaan 4 — Hubungan Sentimen dan Topik (hal. 3)

"Bagaimana hubungan antara hasil analisis sentimen dan pemodelan topik?"

Jawaban Ideal: Dua pendekatan yang bisa digunakan:

Pendekatan A - Sequential (lebih baik):

1. Klasifikasi sentimen → dapatkan label per tweet
2. Pisahkan corpus berdasarkan sentimen
3. Lakukan BERTopic terpisah untuk:
 - Corpus positif → "Topik apa yang disukai pengguna?"
 - Corpus negatif → "Topik apa yang dikritik?"
 - Corpus netral → "Topik apa yang dibahas secara objektif?"

Pendekatan B - Parallel:

1. BERTopic pada seluruh data → identifikasi topik
2. Analisis distribusi sentimen per topik

Referensi: Muhammad Rayhan Nur et al. (2025) di Tabel 1 menggunakan pendekatan ini.

Pertanyaan 5 — Kontribusi Praktis (hal. 3-4)

■ *"Jika tidak membahas faktor yang memengaruhi opini, apa kontribusi praktis penelitian ini?"*

Jawaban Ideal:

- **Untuk developer Roblox:** Mengetahui aspek game yang perlu diperbaiki
 - **Untuk komunitas:** Memahami isu yang paling sering dibahas
 - **Untuk peneliti:** Baseline dataset sentimen Roblox Indonesia
 - **Rekomendasi:** Tambahkan deployment plan seperti dashboard interaktif
-

BAB II - TINJAUAN PUSTAKA

Pertanyaan 6 — Karakteristik Platform X (hal. 5)

■ *"Berapa batas karakter maksimum di Platform X saat ini?"*

Jawaban Ideal:

- Standard user: **280 karakter** (naik dari 140 sejak 2017)
 - Premium/Verified user: hingga **25.000 karakter**
 - **Implikasi:** Data tweet umumnya pendek, cocok untuk BERTopic yang kuat di short text
 - Pre-processing perlu mempertimbangkan panjang teks yang bervariasi
-

Pertanyaan 7 — Perbedaan IndoBERT vs IndoBERTweet (hal. 12-13)

"Apa perbedaan utama antara IndoBERT dan IndoBERTweet?"

Jawaban Ideal: Berdasarkan paper asli [Koto et al. \(2021\)](#):

Aspek	IndoBERT	IndoBERTweet
Training Data	Wikipedia + news Indonesia	409 juta tweet Indonesia
Vocabulary	Formal Indonesian	Expanded dengan slang Twitter
Target	General Indonesian NLP	Social media text
Performance (SmSA)	~85%	90.4%
Keunggulan	Teks formal	Singkatan, emotikon, hashtag

Mengapa IndoBERTweet lebih cocok:

- Vocabulary khusus mengenali @mention, #hashtag, emotikon
- Dilatih dari data Twitter langsung
- Menangani bahasa informal lebih baik

Pertanyaan 8 — Pemahaman BERTopic (hal. 15-16)

"Bagaimana BERTopic menangani dokumen outlier/noise?"

Jawaban Ideal: Berdasarkan [dokumentasi BERTopic](#):

- Outlier ditandai sebagai **Topic -1**
- Penyebab: dokumen tidak cukup dekat dengan cluster manapun
- Cara mengurangi outlier:
 1. Turunkan `min_samples` di HDBSCAN
 2. Gunakan `reduce_outliers()` method
 3. Assign outlier ke topik terdekat dengan threshold

Persentase acceptable: Umumnya < 15-20% outlier. Jika lebih tinggi, perlu tuning parameter.

Pertanyaan 9 — Keterbatasan BERTopic: Single Topic Assignment (hal. 16)

"Bagaimana jika tweet membahas dua topik sekaligus?"

Jawaban Ideal:

Ini memang **keterbatasan utama BERTopic** dibandingkan dengan LDA (Latent Dirichlet Allocation). BERTopic menggunakan **hard clustering** dimana setiap dokumen hanya di-assign ke **satu topik**

saja, padahal dalam realita banyak tweet yang membahas multiple topics.

Contoh Tweet dengan Multiple Topics (Roblox Indonesia):

No	Tweet	Topik 1	Topik 2	Topik 3
1	"Update Roblox terbaru bikin lag parah, mana harga robux makin mahal!"	Update/Patch	Performance/ Lag	Harga/ Ekonomi
2	"Server Roblox down lagi, padahal lagi asik main sama temen"	Server Issues	Social/Friends	-
3	"Beli robux pake gopay error terus, supportnya lama bales"	Pembayaran	Customer Service	-
4	"Game baru di Roblox grafisnya bagus tapi bikin HP panas"	Game Content	Device Performance	-
5	"Event Halloween Roblox seru tapi sayang itemnya mahal banget"	Event/ Seasonal	Harga/Ekonomi	-
6	"Anak gw kecanduan Roblox, duitnya abis buat robux mulu"	Parenting	Spending/ Money	Addiction
7	"Developer game ini curang, robux hasil kerja keras ilang gara-gara bug"	Developer Issues	Bug/Glitch	Ekonomi
8	"Roblox lebih seru dari FF, tapi sayang sering maintenance"	Game Comparison	Server/ Maintenance	-

Dampak pada Hasil Analisis:

Ketika BERTopic memproses tweet #1 "Update Roblox terbaru bikin lag parah, mana harga robux makin mahal!", model hanya akan assign tweet ini ke **SATU** topik, misalnya:

- Jika di-assign ke "Update/Patch" → informasi tentang "lag" dan "harga robux" hilang
- Jika di-assign ke "Performance" → informasi tentang "update" dan "harga" hilang

Ilustrasi Perbedaan LDA vs BERTopic:

Tweet: "Server down lagi, padahal lagi event Halloween"

LDA (Soft Assignment - Multiple Topics):

```
Topik "Server Issues" : 0.55 (55%)
Topik "Event/Seasonal" : 0.35 (35%)
Topik "Gameplay" : 0.10 (10%)

→ Tweet dianggap membahas 2 topik utama
```

```
BERTopic (Hard Assignment - Single Topic):

Topik "Server Issues" : 1.00 (100%) ← ASSIGNED
Topik "Event/Seasonal" : 0.00 (0%)
Topik lainnya : 0.00 (0%)

→ Informasi tentang "event Halloween" HILANG
```

Solusi untuk Mengatasi Keterbatasan:

1. Soft Clustering dengan Probabilitas:

```
from bertopic import BERTopic

# Aktifkan perhitungan probabilitas
topic_model = BERTopic(calculate_probabilities=True)
topics, probs = topic_model.fit_transform(documents)

# Lihat distribusi probabilitas untuk satu dokumen
doc_index = 0
doc_probs = probs[doc_index]

# Ambil top-3 topik dengan probabilitas tertinggi
top_topics = sorted(enumerate(doc_probs), key=lambda x: x[1], reverse=True)[:3]
print(f"Tweet: {documents[doc_index]}")
for topic_id, prob in top_topics:
    if prob > 0.1: # threshold 10%
        print(f"  Topik {topic_id}: {prob:.2%}")
```

Output contoh:

```
Tweet: "Update Roblox bikin lag, robux juga mahal"
Topik 3 (Update): 45.2%
Topik 7 (Performance): 32.1%
Topik 12 (Harga): 18.5%
```

2. Hierarchical Topics:

```
# Lihat relasi antar topik
hierarchical_topics = topic_model.hierarchical_topics(documents)

# Visualisasi hierarki
fig = topic_model.visualize_hierarchy()
fig.show()
```

3. Analisis Per-Sentimen (Rekomendasi):

Lakukan topic modeling terpisah untuk setiap kategori sentimen:

```
# Pisahkan corpus berdasarkan sentimen
positive_docs = [doc for doc, sent in zip(documents, sentiments) if sent == 'positive']
negative_docs = [doc for doc, sent in zip(documents, sentiments) if sent == 'negative']
neutral_docs = [doc for doc, sent in zip(documents, sentiments) if sent == 'neutral']

# Train BERTopic terpisah
model_pos = BERTopic().fit(positive_docs)
model_neg = BERTopic().fit(negative_docs)
model_neu = BERTopic().fit(neutral_docs)
```

Yang Harus Disebutkan di Batasan Penelitian:

"Penelitian ini menggunakan BERTopic yang menerapkan hard clustering, dimana setiap tweet hanya di-assign ke satu topik. Pada kenyataannya, beberapa tweet mungkin membahas lebih dari satu topik secara bersamaan. Keterbatasan ini dimitigasi dengan menggunakan parameter `calculate_probabilities=True` untuk melihat distribusi probabilitas topik pada setiap dokumen."

Pertanyaan 10 — Novelty vs Randy Suryono (2025) (hal. 2, 20)

"Apa yang membedakan penelitian ini dari Randy Suryono (2025) yang juga menganalisis Roblox dengan IndoBERT?"

Jawaban Ideal:

Aspek	Randy Suryono (2025)	Penelitian Ini
Sumber Data	Review Play Store	Tweet Platform X
Model	IndoBERT	IndoBERTweet
Topik	Tidak ada	BERTopic
Konteks	Review produk	Komunitas diskusi
Handling Imbalance	SMOTE	(perlu dijelaskan)

Novelty yang valid:

- 1. Data dari Platform X (bukan Play Store)
- 2. Kombinasi sentimen + topic modeling
- 3. Fokus pada komunitas spesifik

BAB III - METODOLOGI PENELITIAN

Pertanyaan 11 ♦ Etika Scraping (hal. 28)

"Mengapa menggunakan Selenium bukan Twitter API? Bagaimana aspek etisnya?"

Jawaban Ideal: Berdasarkan [arXiv paper on web scraping ethics](#):

Alasan scraping vs API:

- Twitter API berbayar: \$100-\$42.000/bulan sejak 2023
- Basic tier: hanya 10.000 tweet/bulan
- Biaya tidak terjangkau untuk skripsi

Pertimbangan etis yang harus disebutkan:

1. ✓ Data publik (bukan DM atau akun private)
2. ✓ Tidak menyimpan data identitas pengguna
3. ✓ Untuk tujuan akademik non-komersial
4. ✓ Tidak melakukan scraping berlebihan (rate limit)
5. ⚠ Sebutkan di **keterbatasan** bahwa metode ini tidak sesuai ToS X

Kasus hukum: X Corp. v. Bright Data (2024) - pengadilan California menolak argumen X bahwa scraping data publik melanggar hukum.

Pertanyaan 12 ♦ Proses Pelabelan Manual (hal. 29)

"Siapa yang melakukan pelabelan? Bagaimana mengukur konsistensi?"

Jawaban Ideal: Berdasarkan best practices dari [PMC](#):

Yang seharusnya dilakukan:

1. **Minimal 2-3 labeler** untuk subset data (10-20%)
2. **Hitung Cohen's Kappa (κ):**

$$\kappa = (P_o - P_e) / (1 - P_e)$$

- P_o = observed agreement
- P_e = expected agreement by chance

3. **Interpretasi Kappa (Landis & Koch, 1977):**

Nilai κ	Interpretasi
0.81-1.00	Almost perfect
0.61-0.80	Substantial (acceptable)

Jawaban Ideal: Pendekatan **selective normalization** bisa dijustifikasi:

- "gw" → "saya": Menyeragamkan pronouns
- "njing" tetap: Kata ini **mengandung sentiment negatif** yang penting untuk klasifikasi

Rekomendasi:

- Buat **kamus normalisasi** yang eksplisit
- Dokumentasikan kriteria: "Kata yang dinormalisasi adalah kata yang tidak mengandung sentiment"
- Konsistenkan penerapan (jika "gw" → "saya", maka "lo" → "kamu" juga)

Pertanyaan 16 Penanganan Class Imbalance (hal. 32)

"Bagaimana jika distribusi sentimen tidak seimbang?"

Jawaban Ideal: Berdasarkan [Springer](#) dan [MachineLearningMastery](#):

Teknik yang direkomendasikan:

Metode	Kelebihan	Kekurangan
SMOTE	Membuat synthetic samples	Risiko overfitting
Random Oversampling	Simple	Duplikasi data
Random Undersampling	Mengurangi data	Kehilangan informasi
Class Weights	Tidak mengubah data	Perlu tuning

Untuk IndoBERTweet:

```
from transformers import Trainer, TrainingArguments

# Gunakan class_weight dalam loss function
training_args = TrainingArguments(
    ...
    label_smoothing_factor=0.1, # Regularization
)

# Atau gunakan weighted loss
class_weights = torch.tensor([1.0, 2.0, 1.5]) # [netral, negatif, positif]
```

Randy Suryono (2025) menggunakan **SMOTE** dan mendapatkan peningkatan performa signifikan.

Pertanyaan 17 Hyperparameter Fine-tuning (hal. 33-34) ☆ KRITIS

"Berapa nilai learning rate, batch size, dan epoch yang akan digunakan?"

Jawaban Ideal: Berdasarkan [Stanford CS224N](#), [Medium](#), dan best practices:

Nilai yang Direkomendasikan untuk IndoBERTweet:

Parameter	Nilai Rekomendasi	Alasan
Learning Rate	2e-5 hingga 5e-5	Terlalu besar → destroy pretrained weights
Batch Size	16 atau 32	Tergantung GPU memory
Epochs	3-4	BERT sudah pretrained, tidak perlu lama
Warmup Steps	10% dari total steps	Prevent early divergence
Dropout	0.1-0.3	Regularization
Optimizer	AdamW	Standard untuk transformer
Max Length	128 atau 256	Tweet pendek, 128 cukup

Contoh konfigurasi:

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)
```

Strategi Tuning:

- Gunakan **grid search** atau **Optuna** untuk mencari kombinasi optimal
- Monitor **validation loss** untuk early stopping

Pertanyaan 18 Threshold Evaluasi Model (hal. 34-36) ☆ KRITIS

"Berapa nilai minimum accuracy/F1-score yang acceptable?"

Jawaban Ideal: Berdasarkan [GetFocal](#) dan benchmark penelitian terdahulu:

Target Minimum yang Realistis:

Metrik	Minimum Acceptable	Good	Excellent
Accuracy	≥ 80%	≥ 85%	≥ 90%
F1-Score (Weighted)	≥ 0.75	≥ 0.80	≥ 0.85
Precision	≥ 0.75	≥ 0.80	≥ 0.85
Recall	≥ 0.75	≥ 0.80	≥ 0.85

Benchmark dari penelitian terdahulu:

- Damayanti (2025) - IndoBERTweet #BTSComeback: **95% accuracy**
- Nugraha & Febriani (2025) - IndoBERTweet #KaburAjaDulu: **>94%** semua metrik
- Maulana & Lhaksmana (2023) - IndoBERTweet Kanjuruhan: **88% accuracy**

Jika tidak tercapai, langkah yang harus dilakukan:

1. Periksa kualitas pelabelan
2. Tuning hyperparameter
3. Tambah data training
4. Coba data augmentation
5. Gunakan SMOTE untuk imbalance

Pertanyaan 19🔹 Parameter BERTopic (hal. 36-40) ☆ KRITIS

| *"Bagaimana jika BERTopic menghasilkan terlalu banyak atau terlalu sedikit topik?"*

Jawaban Ideal: Berdasarkan [BERTopic Parameter Tuning](#):

Parameter Kunci yang Harus Ditentukan:

```
from bertopic import BERTopic
from umap import UMAP
from hdbscan import HDBSCAN

# UMAP parameters
umap_model = UMAP(
    n_neighbors=15,      # 5-50, lebih kecil = lebih lokal
    n_components=5,      # dimensi reduksi
    min_dist=0.0,        # 0.0 baik untuk clustering
    metric='cosine',
    random_state=42      # reproducibility!
)

# HDBSCAN parameters
hdbscan_model = HDBSCAN(
    min_cluster_size=15,  # minimum dokumen per topik
    min_samples=10,      # mengurangi noise/outlier
    metric='euclidean',
    cluster_selection_method='eom', # atau 'leaf' untuk lebih banyak topik
    prediction_data=True
)
```

```
# BERTopic
topic_model = BERTopic(
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    language="indonesian",
    calculate_probabilities=True,
    verbose=True
)
```

Tuning Strategy:

Masalah	Parameter	Solusi
Terlalu banyak topik	min_cluster_size	Naikkan (20-50)
Terlalu sedikit topik	min_cluster_size	Turunkan (5-10)
Banyak outlier (Topic -1)	min_samples	Turunkan
Topik tidak koheren	n_neighbors	Turunkan (5-10)

Jumlah topik ideal: Tergantung dataset, umumnya **10-30 topik** untuk 2.000-3.000 dokumen.

Pertanyaan 20💎 Validasi Interpretasi Topik (hal. 39-40)

"Siapa yang melakukan interpretasi topik? Ada validasi dari expert?"

Jawaban Ideal:

Evaluasi topic model harus menggunakan kombinasi **metode kuantitatif** (automated metrics) dan **metode kualitatif** (human judgment). Berikut penjelasan lengkap:

A. METODE KUANTITATIF (Automated Metrics)

1. Topic Coherence Metrics

Topic coherence mengukur seberapa koheren kata-kata dalam satu topik berdasarkan co-occurrence (kemunculan bersama) dalam corpus. Semakin sering kata-kata dalam satu topik muncul bersama dalam dokumen, semakin koheren topik tersebut.

a) NPMI — Normalized Pointwise Mutual Information

Nama Lengkap: Normalized Pointwise Mutual Information

Deskripsi: NPMI mengukur seberapa sering dua kata muncul bersama dibandingkan dengan

kemunculan individual mereka, dinormalisasi agar nilainya berada dalam rentang -1 hingga +1. Metrik ini mengoreksi bias terhadap kata-kata yang sangat umum.

Formula:

$$\text{PMI}(w_i, w_j) = \log \left[\frac{P(w_i, w_j)}{P(w_i) \times P(w_j)} \right]$$

$$\text{NPMI}(w_i, w_j) = \text{PMI}(w_i, w_j) / (-\log P(w_i, w_j))$$

Dimana:

- $P(w_i)$ = probabilitas kata w_i muncul dalam corpus
- $P(w_j)$ = probabilitas kata w_j muncul dalam corpus
- $P(w_i, w_j)$ = probabilitas kata w_i dan w_j muncul bersama (dalam window yang sama)

Rentang Nilai: -1 hingga +1

- **+1** = kata selalu muncul bersama (perfect association)
- **0** = kata muncul secara independen (no association)
- **-1** = kata tidak pernah muncul bersama (negative association)

Threshold:

Kategori	Nilai NPMI	Interpretasi
Poor	< 0.0	Topik tidak koheren
Acceptable	0.0 - 0.1	Topik cukup koheren
Good	0.1 - 0.2	Topik koheren
Excellent	> 0.2	Topik sangat koheren

Contoh Perhitungan:

Topik: [roblox, game, main, server, lag]

Misalkan dalam corpus 1000 dokumen:

- "roblox" muncul di 200 dokumen $\rightarrow P(\text{roblox}) = 0.20$
- "game" muncul di 300 dokumen $\rightarrow P(\text{game}) = 0.30$
- "roblox" dan "game" muncul bersama di 150 dokumen $\rightarrow P(\text{roblox}, \text{game}) = 0.15$

$$\begin{aligned}\text{PMI}(\text{roblox}, \text{game}) &= \log(0.15 / (0.20 \times 0.30)) \\ &= \log(0.15 / 0.06) \\ &= \log(2.5) \\ &= 0.916\end{aligned}$$

$$\begin{aligned}\text{NPMI}(\text{roblox}, \text{game}) &= 0.916 / (-\log(0.15)) \\ &= 0.916 / 1.897 \\ &= 0.483\end{aligned}$$

Hasil: NPMI = 0.483 $\rightarrow$ Excellent (kata sangat terkait)

Kelebihan:

- Tidak bias terhadap kata yang sangat umum
- Rentang nilai yang terbatas (-1 hingga +1) mudah diinterpretasi
- Didukung langsung oleh BERTopic dan Gensim

Kekurangan:

- Memerlukan corpus referensi yang cukup besar
- Sensitif terhadap ukuran window co-occurrence

b) C_V — Coherence V (Vector-based)

Nama Lengkap: Coherence V (Vector Space Coherence)

Deskripsi: C_V adalah metrik paling komprehensif yang menggabungkan NPMI dengan cosine similarity. Metrik ini membuat vektor konteks untuk setiap kata berdasarkan co-occurrence, kemudian mengukur kemiripan antar vektor.

Formula (Simplified):

```
C_V = rata-rata cosine_similarity(vektor_kata_i, vektor_kata_j)

dimana vektor setiap kata dibangun dari NPMI dengan semua kata lain dalam topik
```

Rentang Nilai: 0 hingga 1

Threshold:

Kategori	Nilai C_V	Interpretasi
Poor	< 0.3	Topik buruk
Acceptable	0.3 - 0.5	Topik cukup
Good	0.5 - 0.7	Topik baik
Excellent	> 0.7	Topik sangat baik

Kelebihan:

- Paling komprehensif, mempertimbangkan relasi multi-kata
- Berkorelasi baik dengan penilaian manusia

Kekurangan:

- Sensitif terhadap noise dan data kotor
- Komputasi lebih berat

c) C_UCI — UCI Coherence

Nama Lengkap: UCI Coherence (dari University of California, Irvine)

Deskripsi: C_UCI menggunakan PMI dengan sliding window pada corpus eksternal (biasanya Wikipedia). Setiap pasangan kata dihitung PMI-nya berdasarkan co-occurrence dalam window tertentu.

Formula:

$$C_UCI = (2 / (N \times (N-1))) \times \sum PMI(w_i, w_j)$$

dimana N = jumlah kata dalam topik, dan PMI dihitung dengan sliding window

Rentang Nilai: Negatif hingga positif (tidak terbatas)

Kelebihan:

- Menggunakan corpus eksternal yang besar (Wikipedia)
- Tidak tergantung pada corpus penelitian

Kekurangan:

- Wikipedia berbahasa Indonesia lebih kecil dari English
- Tidak cocok untuk domain spesifik atau bahasa informal (slang Twitter)

d) C_UMass — UMass Coherence

Nama Lengkap: UMass Coherence (dari University of Massachusetts)

Deskripsi: C_UMass mengukur seberapa sering kata-kata muncul bersama dalam dokumen yang sama, menggunakan corpus internal (dokumen penelitian sendiri). Tidak memerlukan corpus eksternal.

Formula:

$$C_UMass(w_i, w_j) = \log [(D(w_i, w_j) + 1) / D(w_j)]$$

dimana:

- $D(w_i, w_j)$ = jumlah dokumen yang mengandung kedua kata
- $D(w_j)$ = jumlah dokumen yang mengandung kata w_j

Rentang Nilai: Negatif (mendekati 0 = lebih baik)

Threshold:

Kategori	Nilai UMass	Interpretasi

Poor	< -4.0	Topik buruk
Acceptable	-4.0 to -2.0	Topik cukup
Good	-2.0 to -1.0	Topik baik
Excellent	> -1.0	Topik sangat baik

Contoh Perhitungan:

Topik: [roblox, game, main] Dalam corpus 100 dokumen: - Dokumen mengandung "roblox" dan "game": 25 dokumen - Dokumen mengandung "game": 40 dokumen $C_UMass(roblox, game) = \log((25 + 1) / 40)$ $= \log(26 / 40)$ $= \log(0.65)$ $= -0.43$ Hasil: UMass = -0.43 → Excellent (kata sering muncul bersama)

Kelebihan:

- Tidak memerlukan corpus eksternal
- Cepat dihitung

Kekurangan:

- Bisa bias jika corpus penelitian kecil
- Tidak menangkap relasi semantik di luar corpus

Ringkasan Perbandingan Metrik Coherence:

Metrik	Nama Lengkap	Corpus	Rentang	Target	Rekomendasi
NPMI	Normalized Pointwise Mutual Information	Internal/ Eksternal	-1 to +1	≥ 0.1	Direkomendasikan
C_V	Coherence Vector	Internal	0 to 1	≥ 0.5	Alternatif
C_UCI	UCI Coherence	Eksternal (Wikipedia)	Negatif- Positif	Varies	Untuk English

C_UMass	UMass Coherence	Internal	Negatif	≥ -2.0	Corpus kecil
---------	--------------------	----------	---------	-------------	--------------

Rekomendasi untuk BERTopic Indonesia: Gunakan **NPMI** karena paling robust, tidak bias, dan didukung langsung oleh BERTopic.

Implementasi Python:

```
from bertopic import BERTopic
from gensim.models.coherencemodel import CoherenceModel
from gensim.corpora import Dictionary

# Setelah training BERTopic
topics = topic_model.get_topics()
topic_words = [[word for word, _ in topic_model.get_topic(topic_id)]
               for topic_id in range(len(topics)-1)] # exclude -1 (outlier)

# Buat dictionary dan corpus untuk Gensim
tokenized_docs = [doc.split() for doc in documents]
dictionary = Dictionary(tokenized_docs)
corpus = [dictionary.doc2bow(doc) for doc in tokenized_docs]

# Hitung berbagai metrik coherence
for metric in ['c_npmi', 'c_v', 'u_mass']:
    coherence_model = CoherenceModel(
        topics=topic_words,
        texts=tokenized_docs,
        dictionary=dictionary,
        coherence=metric
    )
    score = coherence_model.get_coherence()
    print(f"{metric.upper()}: {score:.4f}")
```

Referensi:

- [Röder et al. \(2015\) - Exploring the Space of Topic Coherence Measures](#)
 - [Baeldung - When Coherence Score Is Good or Bad](#)
 - [Gensim CoherenceModel Documentation](#)
-

2. Topic Diversity Metrics

Topic diversity mengukur seberapa berbeda/unik topik-topik yang dihasilkan. Model yang baik harus menghasilkan topik-topik yang berbeda satu sama lain, bukan topik yang redundan/mirip.

a) TD — Topic Diversity

Nama Lengkap: Topic Diversity

Deskripsi: TD mengukur persentase kata unik di antara semua kata teratas dari semua topik. Jika banyak kata yang sama muncul di berbagai topik, diversity rendah (topik redundan).

Formula:

$$TD = \frac{|\text{unique words in top-N of all topics}|}{(K \times N)}$$

dimana:

- K = jumlah topik
- N = jumlah kata teratas per topik (biasanya 25)

Rentang Nilai: 0 hingga 1

- **1.0** = semua kata unik (tidak ada duplikasi antar topik)
- **0.0** = semua topik memiliki kata yang sama

Threshold:

Kategori	Nilai TD	Interpretasi
Poor	< 0.6	Topik sangat redundan
Acceptable	0.6 - 0.8	Topik cukup beragam
Good	0.8 - 0.9	Topik beragam
Excellent	> 0.9	Topik sangat beragam

Contoh Perhitungan:

3 Topik dengan 5 kata teratas masing-masing:

Topik 1: [roblox, game, main, server, lag]

Topik 2: [update, fitur, developer, patch, game] ← "game" duplikat

Topik 3: [robux, beli, mahal, gratis, server] ← "server" duplikat

Total kata: $3 \times 5 = 15$

Kata unik: {roblox, game, main, server, lag, update, fitur, developer, patch, robux, beli, mahal, gratis} = 13

$$TD = 13 / 15 = 0.867$$

Hasil: TD = 0.867 → Good (topik cukup beragam)

b) Inverted RBO — Inverted Rank-Biased Overlap

Nama Lengkap: Inverted Rank-Biased Overlap

Deskripsi: RBO mengukur kemiripan antara dua daftar kata berperingkat. Inverted RBO mengukur ketidakmiripan (diversity). Metrik ini memberikan bobot lebih pada kata-kata di peringkat atas.

Rentang Nilai: 0 hingga 1 (lebih tinggi = lebih beragam)

Target: ≥ 0.7

c) Pairwise Jaccard Distance

Nama Lengkap: Pairwise Jaccard Distance

Deskripsi: Mengukur jarak Jaccard antara setiap pasangan topik, kemudian dirata-ratakan. Jaccard distance = 1 - Jaccard similarity.

Formula:

```
Jaccard(A, B) = |A ∩ B| / |A ∪ B|
Jaccard_Distance(A, B) = 1 - Jaccard(A, B)

Pairwise_Jaccard = rata-rata Jaccard_Distance untuk semua pasangan topik
```

Rentang Nilai: 0 hingga 1 (lebih tinggi = lebih beragam)

Target: ≥ 0.8

Ringkasan Metrik Diversity:

Metrik	Nama Lengkap	Rentang	Target
TD	Topic Diversity	0-1	≥ 0.8
Inverted RBO	Inverted Rank-Biased Overlap	0-1	≥ 0.7
Pairwise Jaccard	Pairwise Jaccard Distance	0-1	≥ 0.8

Implementasi Python:

```
def compute_topic_diversity(topic_model, top_n=25):
    """
    Menghitung Topic Diversity (TD)

    Args:
        topic_model: Model BERTopic yang sudah di-train
        top_n: Jumlah kata teratas per topik (default: 25)

    Returns:
        float: Nilai diversity (0-1)
    """
    topics = topic_model.get_topics()
    all_words = []

    for topic_id in topics:
        if topic_id != -1: # exclude outlier topic
            words = [word for word, _ in topic_model.get_topic(topic_id)[:top_n]]
            all_words.extend(words)

    unique_words = set(all_words)
    num_topics = len([t for t in topics if t != -1])
```

```

    if num_topics == 0:
        return 0.0

    diversity = len(unique_words) / (num_topics * top_n)
    return diversity

# Contoh penggunaan
td_score = compute_topic_diversity(topic_model, top_n=25)
print(f"Topic Diversity: {td_score:.4f}")

# Interpretasi
if td_score >= 0.9:
    print("→ Excellent: Topik sangat beragam")
elif td_score >= 0.8:
    print("→ Good: Topik beragam")
elif td_score >= 0.6:
    print("→ Acceptable: Topik cukup beragam")
else:
    print("→ Poor: Topik redundan, perlu tuning parameter")

```

Referensi:

- [Dieng et al. \(2020\) - Topic Modeling in Embedding Spaces](#)
- [GitHub - Topic Model Diversity](#)

3. Kombinasi Metrik: Topic Quality Score

Untuk evaluasi komprehensif, gabungkan coherence dan diversity menjadi satu skor kualitas:

Formula:

$$\text{Quality Score} = \text{Coherence} \times \text{Diversity}$$

Implementasi:

```

def evaluate_topic_quality(topic_model, documents):
    """
    Menghitung Topic Quality Score (kombinasi coherence dan diversity)
    """
    # Hitung NPMI Coherence
    # ... (gunakan kode coherence di atas)
    coherence_score = 0.15 # contoh hasil

    # Hitung Topic Diversity
    diversity_score = compute_topic_diversity(topic_model)

    # Quality Score
    quality_score = coherence_score * diversity_score

    print("=" * 50)
    print("TOPIC MODEL EVALUATION")
    print("=" * 50)
    print(f"NPMI Coherence   : {coherence_score:.4f} (target ≥ 0.1)")
    print(f"Topic Diversity    : {diversity_score:.4f} (target ≥ 0.8)")
    print(f"Quality Score      : {quality_score:.4f}")
    print("=" * 50)

    return {
        'coherence': coherence_score,

```

```
'diversity': diversity_score,  
'quality': quality_score  
}
```

Referensi: [Dieng et al. \(2019\)](#)

B. METODE KUALITATIF (Human Evaluation)

Metode kualitatif dianggap **gold standard** untuk evaluasi topic model karena melibatkan penilaian manusia secara langsung. Metrik otomatis (NPMI, C_V, dll.) hanya mengukur statistik co-occurrence, sedangkan manusia dapat menilai apakah topik benar-benar **bermakna dan dapat diinterpretasi**.

Referensi Utama: [Chang et al. \(2009\) "Reading Tea Leaves: How Humans Interpret Topic Models"](#) - Paper klasik yang memperkenalkan Word Intrusion dan Topic Intrusion tasks.

1. Word Intrusion Task

Nama Lengkap: Word Intrusion Task (Tugas Deteksi Kata Penyusup)

Referensi: Chang et al. (2009), NIPS

Deskripsi: Word Intrusion mengukur **koherensi semantik internal** suatu topik. Evaluator diminta mengidentifikasi satu kata "penyusup" (intruder) yang disisipkan ke dalam daftar kata-kata dari suatu topik. Jika topik koheren, evaluator akan mudah menemukan kata yang tidak sesuai.

Asumsi: Topik yang baik terdiri dari kata-kata yang secara semantik berhubungan erat. Kata dari topik lain akan terlihat "aneh" atau "tidak nyambung".

Prosedur Lengkap:

Langkah 1: Persiapan Stimulus

- Untuk setiap topik, ambil **5 kata teratas** (highest probability words)
- Pilih **1 kata intruder** secara random dari topik lain (biasanya dari kata peringkat 5-15 topik lain)
- Acak urutan 6 kata tersebut

Langkah 2: Pelaksanaan

- Rekrut **minimal 3 evaluator** (bisa mahasiswa, dosen, atau crowdworker)
- Setiap evaluator melihat 6 kata dan memilih **1 kata yang tidak sesuai**
- Evaluator **TIDAK** diberitahu nama topik atau konteks lainnya

Langkah 3: Perhitungan

```
Model Precision = (Jumlah evaluator yang benar mengidentifikasi intruder) / (Total evaluator)
```

Contoh Word Intrusion (Konteks Roblox Indonesia):

Contoh 1 - Topik "Game Performance":

Kata-kata asli topik: [lag, fps, freeze, server, crash]

Intruder dari topik "Ekonomi": robux

Stimulus yang ditampilkan (diacak):

fps	lag	robux	crash	freeze	server
(A)	(B)	(C)	(D)	(E)	(F)

Pertanyaan: "Manakah kata yang TIDAK SESUAI dengan kata lainnya?"

Jawaban benar: C (robux) - karena robux adalah mata uang game, sedangkan kata lain berkaitan dengan performa teknis.

Contoh 2 - Topik "Update & Fitur":

Kata-kata asli topik: [update, patch, fitur, developer, versi]

Intruder dari topik "Social": teman

Stimulus yang ditampilkan:

patch	teman	update	fitur	versi	developer
(A)	(B)	(C)	(D)	(E)	(F)

Jawaban benar: B (teman)

Contoh 3 - Topik "Harga & Ekonomi":

Kata-kata asli topik: [robux, mahal, beli, gratis, diskon]

Intruder dari topik "Performance": lag

Stimulus yang ditampilkan:

mahal	diskon	lag	robux	beli	gratis
(A)	(B)	(C)	(D)	(E)	(F)

Jawaban benar: C (lag)

Interpretasi Model Precision:

Model Precision	Kategori	Interpretasi
> 90%	Excellent	Topik sangat koheren, kata-kata sangat terkait
80% - 90%	Good	Topik koheren, mudah diinterpretasi
70% - 80%	Acceptable	Topik cukup koheren

60% - 70%	Fair	Topik kurang koheren, perlu review
< 60%	Poor	Topik tidak koheren, perlu tuning ulang

Target untuk skripsi: Model Precision $\geq 70\%$

Implementasi Form Evaluasi:

FORM EVALUASI WORD INTRUSION

Evaluator: _____ Tanggal: _____

Instruksi:

Untuk setiap kelompok kata di bawah, pilih SATU kata yang menurut Anda TIDAK SESUAI atau TIDAK BERHUBUNGAN dengan kata lainnya.

Kelompok 1:

☐ A. fps ☐ B. lag ☐ C. robux
☐ D. crash ☐ E. freeze ☐ F. server

Kelompok 2:

☐ A. patch ☐ B. teman ☐ C. update
☐ D. fitur ☐ E. versi ☐ F. developer

Kelompok 3:

☐ A. mahal ☐ B. diskon ☐ C. lag
☐ D. robux ☐ E. beli ☐ F. gratis

Perhitungan dan Pelaporan:

```
# Data hasil evaluasi Word Intrusion
# 1 = benar mengidentifikasi intruder, 0 = salah

word_intrusion_results = {
    'Topik 1': {'E1': 1, 'E2': 1, 'E3': 1}, # 3/3 benar
    'Topik 2': {'E1': 1, 'E2': 0, 'E3': 1}, # 2/3 benar
    'Topik 3': {'E1': 1, 'E2': 1, 'E3': 0}, # 2/3 benar
    'Topik 4': {'E1': 0, 'E2': 0, 'E3': 1}, # 1/3 benar
    # ... dst
}

# Hitung Model Precision per topik
for topic, results in word_intrusion_results.items():
    precision = sum(results.values()) / len(results)
    print(f"{topic}: {precision:.2%}")

# Hitung rata-rata keseluruhan
all_scores = [sum(r.values())/len(r) for r in word_intrusion_results.values()]
avg_precision = sum(all_scores) / len(all_scores)
print(f"\nAverage Model Precision: {avg_precision:.2%}")
```

Output Pelaporan:

Tabel X. Hasil Word Intrusion Task

Topik	Kata Asli	Intruder	E1	E2	E3	Precision
1	lag, fps, freeze, server, crash	robux	✓	✓	✓	100%
2	update, patch, fitur, developer, versi	teman	✓	✗	✓	67%
3	robux, mahal, beli, gratis, diskon	lag	✓	✓	✗	67%
...	...	...	...	...	...	...

Average Model Precision: 78% (Acceptable)

Referensi:

- [Chang et al. \(2009\) - Reading Tea Leaves](#)
- [Lau et al. \(2014\) - Machine Reading Tea Leaves](#)

2. Topic Intrusion Task

Nama Lengkap: Topic Intrusion Task (Tugas Deteksi Topik Penyusup)

Referensi: Chang et al. (2009), NIPS

Deskripsi: Topic Intrusion mengukur seberapa baik **topik-topik yang di-assign ke dokumen** sesuai dengan isi dokumen tersebut. Evaluator melihat dokumen beserta 4 topik (3 topik dengan probabilitas tinggi + 1 topik intruder), lalu memilih topik yang **tidak sesuai** dengan dokumen.

Perbedaan dengan Word Intrusion:

- Word Intrusion: menguji koherensi **internal** topik (apakah kata-kata dalam topik berhubungan)
- Topic Intrusion: menguji koherensi **eksternal** (apakah topik sesuai dengan dokumen)

Prosedur Lengkap:

Langkah 1: Persiapan Stimulus

- Pilih dokumen (tweet) secara random
- Ambil **3 topik dengan probabilitas tertinggi** untuk dokumen tersebut
- Pilih **1 topik intruder** secara random (topik dengan probabilitas rendah atau nol)
- Setiap topik direpresentasikan dengan **8 kata teratas**
- Acak urutan 4 topik tersebut

Langkah 2: Pelaksanaan

- Tampilkan dokumen (tweet) + 4 topik
- Evaluator memilih **1 topik yang TIDAK SESUAI** dengan dokumen

Contoh Topic Intrusion (Konteks Roblox Indonesia):

TOPIC INTRUSION TASK

DOKUMEN:

"Roblox update terbaru bikin game lag parah, fps drop terus, sampe ga bisa main dengan lancar"

Pilih SATU topik yang TIDAK SESUAI dengan dokumen di atas:

[] TOPIK A: lag, fps, freeze, crash, server, loading, koneksi, error
(Performance Issues)

[] TOPIK B: update, patch, versi, fitur, developer, maintenance, download, changelog
(Update & Maintenance)

[] TOPIK C: robux, mahal, beli, harga, diskon, gratis, promo, bayar
(Ekonomi & Harga) ← INTRUDER

[] TOPIK D: main, game, seru, asik, player, gaming, experience, gameplay
(Gameplay Experience)

Jawaban benar: TOPIK C (Ekonomi & Harga)

Alasan: Dokumen membahas masalah teknis (lag, fps) dan update, tidak membahas harga atau pembelian robux.

Contoh Lain:

Dokumen 2:

Tweet: "Beli robux 100rb tapi ga masuk-masuk, support Roblox lama banget balesnya"

Topik yang ditampilkan:

- | | |
|--|-----------------------------|
| A. robux, mahal, beli, harga, bayar, transaksi | (Ekonomi) ← SESUAI |
| B. support, customer, komplain, refund, respon | (Customer Service) ← SESUAI |
| C. lag, fps, freeze, crash, server | (Performance) ← INTRUDER |
| D. error, bug, masalah, gagal, tidak bisa | (Technical Issues) ← SESUAI |

Jawaban benar: C (Performance) - tidak ada pembahasan lag/fps

Dokumen 3:

Tweet: "Event Halloween Roblox tahun ini itemnya bagus-bagus, sayang harganya mahal"

Topik yang ditampilkan:

- | | |
|---|----------------------------|
| A. event, halloween, christmas, seasonal, special | (Events) ← SESUAI |
| B. item, skin, kostum, avatar, accessories | (Items/Cosmetics) ← SESUAI |
| C. server, maintenance, down, offline | (Server Issues) ← INTRUDER |
| D. mahal, harga, robux, beli, diskon | (Ekonomi) ← SESUAI |

Jawaban benar: C (Server Issues)

Interpretasi Topic Log Odds:

Topic Log Odds	Kategori	Interpretasi
----------------	----------	--------------

> 2.0	Excellent	Topik assignment sangat sesuai dengan dokumen
1.5 - 2.0	Good	Topik assignment sesuai
1.0 - 1.5	Acceptable	Topik assignment cukup sesuai
< 1.0	Poor	Topik assignment kurang sesuai

Target untuk skripsi: Topic Log Odds ≥ 1.0 atau Model Precision $\geq 70\%$

Implementasi Python:

```
import numpy as np

# Hasil evaluasi: 1 = benar identifikasi intruder, 0 = salah
topic_intrusion_results = [
    {'doc_id': 1, 'E1': 1, 'E2': 1, 'E3': 1}, # semua benar
    {'doc_id': 2, 'E1': 1, 'E2': 0, 'E3': 1}, # 2/3 benar
    {'doc_id': 3, 'E1': 0, 'E2': 1, 'E3': 1}, # 2/3 benar
    # ... dst untuk semua dokumen yang dievaluasi
]

# Hitung Model Precision
precisions = []
for result in topic_intrusion_results:
    scores = [result['E1'], result['E2'], result['E3']]
    precision = sum(scores) / len(scores)
    precisions.append(precision)
    print(f"Doc {result['doc_id']}: {precision:.2%}")

avg_precision = np.mean(precisions)
print(f"\nAverage Topic Intrusion Precision: {avg_precision:.2%}")
```

Referensi:

- [Chang et al. \(2009\) - NIPS](#)
 - [ACL Anthology - Topic Intrusion for Automatic Evaluation](#)
-

3. Expert Topic Labeling

Nama Lengkap: Expert Topic Labeling with Inter-Rater Agreement (Pelabelan Topik oleh Expert dengan Kesepakatan Antar-Penilai)

Referensi: Mimno et al. (2011), EMNLP

Deskripsi: Metode ini mengukur apakah topik yang dihasilkan model dapat **diberi label/nama yang konsisten** oleh beberapa evaluator independen. Jika topik jelas dan koheren, evaluator yang berbeda akan memberikan label yang serupa.

Prosedur Lengkap:

Langkah 1: Persiapan

- Siapkan daftar semua topik dari BERTopic
- Setiap topik direpresentasikan dengan **10-15 kata teratas**
- Rekrut **minimal 2-3 evaluator** yang independen
- Siapkan **daftar kategori label** (bisa open-ended atau predefined)

Langkah 2: Pelaksanaan

- Setiap evaluator **secara independen** memberikan label untuk setiap topik
- Evaluator tidak boleh berdiskusi atau melihat jawaban evaluator lain
- Bisa menggunakan:
 - ☐ **Open labeling**: Evaluator bebas memberikan nama topik
 - ☐ **Closed labeling**: Evaluator memilih dari daftar kategori yang sudah ditentukan

Langkah 3: Perhitungan Agreement

Contoh Expert Topic Labeling (Konteks Roblox Indonesia):

FORM PELABELAN TOPIK OLEH EXPERT	
Evaluator: _____	Tanggal: _____
Instruksi: Berikan SATU label/nama yang paling tepat untuk setiap kelompok kata di bawah ini.	
TOPIK 1: [lag, fps, freeze, server, crash, ping, loading, error, disconnect, timeout]	
Label Anda: _____	
TOPIK 2: [robux, mahal, beli, harga, gratis, diskon, promo, bayar, murah, duit]	
Label Anda: _____	
TOPIK 3: [update, patch, fitur, versi, developer, maintenance, download, changelog, rilis, baru]	
Label Anda: _____	

Hasil Pelabelan (Contoh):

Topik	Kata-kata	Evaluator 1	Evaluator 2	Evaluator 3
-------	-----------	-------------	-------------	-------------

1	lag, fps, freeze...	"Masalah Performa"	"Performa Game"	"Technical Issues"
2	robux, mahal, beli...	"Harga & Pembelian"	"Ekonomi Robux"	"Harga"
3	update, patch, fitur...	"Update Game"	"Pembaruan"	"Update & Fitur"
4	teman, main, bareng...	"Sosial/Friends"	"Multiplayer"	"Main Bareng"
5	cheater, hack, banned...	"Kecurangan"	"Cheating"	"Hack & Ban"

Metrik Inter-Rater Agreement:

a) Cohen's Kappa (untuk 2 evaluator):

Nama Lengkap: Cohen's Kappa Coefficient

Formula:

$$\kappa = (P_o - P_e) / (1 - P_e)$$

dimana:

- P_o = Observed agreement (proporsi kesepakatan yang teramati)
- P_e = Expected agreement by chance (proporsi kesepakatan yang diharapkan secara kebetulan)

Contoh Perhitungan:

2 Evaluator menilai 10 topik ke dalam 3 kategori: A, B, C

		Evaluator 2			Total
		A	B	C	
E1	A	3	1	0	4
	B	0	2	1	3
	C	1	0	2	3
Total		4	3	3	10

$$P_o = (3 + 2 + 2) / 10 = 0.70 \quad (\text{agreement pada diagonal})$$

$$P_e = (4 \times 4 + 3 \times 3 + 3 \times 3) / 100 = (16 + 9 + 9) / 100 = 0.34$$

$$\kappa = (0.70 - 0.34) / (1 - 0.34) = 0.36 / 0.66 = 0.545$$

b) Fleiss' Kappa (untuk > 2 evaluator):

Nama Lengkap: Fleiss' Kappa (Generalisasi Cohen's Kappa untuk multiple raters)

```
from statsmodels.stats.inter_rater import fleiss_kappa, aggregate_raters
import numpy as np
```

```
# Data: setiap baris = 1 topik, setiap kolom = 1 evaluator
# Nilai = kategori label (1, 2, 3, ...)
labels = np.array([
    [1, 1, 1], # Topik 1: semua setuju kategori 1
    [2, 2, 3], # Topik 2: 2 evaluator kategori 2, 1 evaluator kategori 3
    [1, 2, 1], # Topik 3: 2 evaluator kategori 1, 1 evaluator kategori 2
    [3, 3, 3], # Topik 4: semua setuju kategori 3
    [2, 2, 2], # Topik 5: semua setuju kategori 2
])

# Konversi ke format Fleiss
table, categories = aggregate_raters(labels)
kappa = fleiss_kappa(table, method='fleiss')

print(f"Fleiss' Kappa: {kappa:.3f}")
```

Interpretasi Kappa:

Nilai Kappa	Kategori	Interpretasi
0.81 - 1.00	Almost Perfect	Evaluator hampir selalu setuju
0.61 - 0.80	Substantial	Target minimum untuk skripsi
0.41 - 0.60	Moderate	Kesepakatan cukup, perlu review kriteria
0.21 - 0.40	Fair	Kesepakatan rendah
< 0.21	Slight/Poor	Evaluator tidak konsisten, kriteria bermasalah

Target untuk skripsi: Cohen's/Fleiss' Kappa ≥ 0.61 (Substantial)

Pelaporan Hasil:

```
Tabel X. Hasil Expert Topic Labeling

| Topik | Top-10 Words | E1 | E2 | E3 | Label Konsensus |
|-----|-----|----|----|----|-----|
| 1 | lag, fps, freeze... | Performa | Performa | Technical | Game Performance |
| 2 | robux, mahal, beli... | Harga | Ekonomi | Harga | Harga & Ekonomi |
| 3 | update, patch... | Update | Pembaruan | Update | Software Update |
| ... | ... | ... | ... | ... | ... |

Inter-Rater Agreement:
- Fleiss' Kappa: 0.72 (Substantial Agreement)
- Persentase Agreement: 85%
```

Referensi:

- [Landis & Koch \(1977\) - Kappa Interpretation Guidelines](#)
- [Fleiss \(1971\) - Measuring Nominal Scale Agreement](#)

4. Topic Rating Task (Human Coherence Judgment)

Referensi Utama: [Newman et al. \(2010\) "Automatic Evaluation of Topic Coherence"](#)

Tujuan: Mengukur apakah evaluator manusia setuju bahwa topik yang dihasilkan model adalah koheren dan bermakna.

Prosedur:

1. Persiapan:

- Siapkan daftar semua topik dari BERTopic (misal: 15 topik)
- Setiap topik direpresentasikan oleh **10 kata teratas**
- Rekrut **minimal 3 evaluator** independen

2. Penilaian dengan Likert Scale:

Setiap evaluator menilai setiap topik menggunakan skala 3-poin:

Skor	Label	Kriteria
3	Useful/Coherent	Topik koheren, bermakna, mudah diberi label
2	Average/Neutral	Topik cukup koheren, sebagian kata relevan
1	Useless/Incoherent	Topik tidak koheren, kata-kata tidak berhubungan

3. Contoh Form Penilaian:

FORM PENILAIAN KOHERENSI TOPIK	
Evaluator: _____	Tanggal: _____
Topik 1: [lag, fps, freeze, server, crash, ping, koneksi, loading, error, disconnect]	
Apakah kata-kata di atas membentuk tema yang koheren?	
☐ 3 (Koheren)	☐ 2 (Cukup) ☐ 1 (Tidak Koheren)
Label yang Anda usulkan: _____	
Topik 2: [update, fitur, baru, developer, patch, versi, download, maintenance, changelog, release]	
Apakah kata-kata di atas membentuk tema yang koheren?	
☐ 3 (Koheren)	☐ 2 (Cukup) ☐ 1 (Tidak Koheren)
Label yang Anda usulkan: _____	

4. Perhitungan Inter-Annotator Agreement (IAA):

Metode A - Krippendorff's Alpha (Direkomendasikan untuk > 2 evaluator dengan skala

ordinal):

```
import krippendorff
import numpy as np

# Data: baris = evaluator, kolom = topik
# Nilai: 1 (Useless), 2 (Average), 3 (Useful)
ratings = np.array([
    [3, 2, 3, 1, 2, 3, 2, 3, 1, 2, 3, 2, 3, 2, 1], # Evaluator 1
    [3, 2, 3, 1, 2, 3, 3, 3, 1, 2, 3, 2, 2, 2, 1], # Evaluator 2
    [3, 3, 3, 1, 2, 3, 2, 3, 2, 2, 3, 2, 3, 2, 1], # Evaluator 3
])

alpha = krippendorff.alpha(ratings, level_of_measurement='ordinal')
print(f"Krippendorff's Alpha: {alpha:.3f}")
```

Metode B - Spearman Correlation (untuk korelasi antar evaluator):

```
from scipy.stats import spearmanr
import numpy as np

# Hitung rata-rata korelasi antar evaluator
correlations = []
for i in range(len(ratings)):
    others = np.mean(np.delete(ratings, i, axis=0), axis=0)
    corr, _ = spearmanr(ratings[i], others)
    correlations.append(corr)

avg_correlation = np.mean(correlations)
print(f"Average Spearman Correlation: {avg_correlation:.3f}")
```

Metode C - Fleiss' Kappa (untuk data kategorikal dengan > 2 evaluator):

```
from statsmodels.stats.inter_rater import fleiss_kappa, aggregate_raters

# Konversi ke format yang dibutuhkan
ratings_transposed = ratings.T # topik x evaluator
table, _ = aggregate_raters(ratings_transposed)
kappa = fleiss_kappa(table, method='fleiss')
print(f"Fleiss' Kappa: {kappa:.3f}")
```

5. Interpretasi Hasil:

Metrik	Poor	Fair	Moderate	Substantial	Excellent
Krippendorff's α	< 0.40	0.40-0.60	0.60-0.67	0.67-0.80	> 0.80
Spearman ρ	< 0.40	0.40-0.55	0.55-0.70	0.70-0.85	> 0.85
Fleiss' κ	< 0.20	0.21-0.40	0.41-0.60	0.61-0.80	> 0.80

Target untuk skripsi: Krippendorff's $\alpha \geq 0.67$ atau Fleiss' $\kappa \geq 0.61$

6. Pelaporan Hasil:

Tabel X. Hasil Evaluasi Koherensi Topik oleh 3 Evaluator

Topik	Kata Kunci	E1	E2	E3	Mean	Label Konsensus
1	lag, fps...	3	3	3	3.00	Game Performance
2	update...	2	3	2	2.33	Software Update
...	...	...	...	...	...	...

Inter-Annotator Agreement:

- Krippendorff's Alpha: 0.72 (Substantial)
- Average Topic Coherence Score: 2.45/3.00 (81.7%)

Referensi Tambahan:

- [Newman et al. \(2010\) - ACL Anthology](#)
- [Mimno et al. \(2011\) - Optimizing Semantic Coherence](#)
- [Lau et al. \(2014\) - Machine Reading Tea Leaves](#)

C. REKOMENDASI EVALUASI UNTUK PROPOSAL INI

Minimum yang harus dilakukan:

No	Evaluasi	Metrik	Target	Prioritas
1	Coherence	NPMI	≥ 0.1	Wajib
2	Diversity	TD	≥ 0.8	Wajib
3	Word Intrusion	Model Precision	$\geq 70\%$	Disarankan
4	Expert Labeling	Cohen's Kappa	≥ 0.61	Disarankan
5	Topic Rating Task	Krippendorff's α	≥ 0.67	Direkomendasikan

Implementasi praktis:

```
# Evaluasi komprehensif BERTopic
def evaluate_topic_model(topic_model, documents):
    results = {}

    # 1. Topic Coherence (NPMI)
    # ... (kode di atas)
    results['npmi'] = coherence_score

    # 2. Topic Diversity
    results['diversity'] = compute_topic_diversity(topic_model)

    # 3. Outlier Ratio
    topic_info = topic_model.get_topic_info()
    outlier_count = topic_info[topic_info['Topic'] == -1]['Count'].values[0]
    total_docs = len(documents)
    results['outlier_ratio'] = outlier_count / total_docs
```



```
# 4. Number of Topics
results['num_topics'] = len(topic_model.get_topics()) - 1 # exclude -1

# Print summary
print("=" * 50)
print("TOPIC MODEL EVALUATION SUMMARY")
print("=" * 50)
print(f"NPMI Coherence : {results['nmpi']:.4f} (target ≥ 0.1)")
print(f"Topic Diversity: {results['diversity']:.4f} (target ≥ 0.8)")
print(f"Outlier Ratio : {results['outlier_ratio']:.2%} (target < 15%)")
print(f"Number of Topics: {results['num_topics']}")
print("=" * 50)

return results
```

Referensi Tambahan:

- [ACL Anthology - Topic Intrusion](#)
- [Gensim Coherence Documentation](#)
- [BERTopic Evaluation Guide](#)
- [Towards Data Science - Topic Coherence](#)

Pertanyaan 21 Kontribusi Bidang Sistem Informasi

"Dimana aspek 'sistem informasi' dalam penelitian ini?"

Jawaban Ideal: Mahasiswa bisa menjawab:

1. **Analisis kebutuhan informasi** komunitas gaming
2. **Text mining** sebagai bagian Business Intelligence
3. **Pengembangan sistem:** Dashboard visualisasi sentimen (jika ada)
4. **Information extraction** dari social media

Rekomendasi: Tambahkan rencana **deployment** seperti:

- Dashboard Streamlit/Power BI
- API endpoint untuk klasifikasi real-time
- Laporan berkala otomatis

B. KESALAHAN KONSEP (TERVERIFIKASI)

1. [PERLU KLARIFIKASI] Periode Pengambilan Data Sangat Singkat

- **Masalah:** Hanya 7 hari (2.465 tweet)
- **Risiko:** Bias temporal, tidak representatif
- **Solusi:** Minimal 4-8 minggu, atau jelaskan sebagai keterbatasan

2. [PERLU KLARIFIKASI] Pelabelan Manual Tanpa Inter-Rater Reliability

- **Masalah:** Tidak disebutkan jumlah labeler dan metode konsistensi
- **Solusi:** Gunakan 2-3 labeler, hitung Cohen's Kappa (target $\kappa \geq 0.60$)

3. [CONFIRMED] Referensi Duplikat dengan Suffix Berbeda

- **Contoh:** Damayanti (2025a, 2025b, 2025c) → semuanya paper yang sama
- **Dampak:** Daftar pustaka tidak rapi, kesan manajemen referensi buruk

4. [CONFIRMED] Tidak Ada Penanganan Class Imbalance

- **Solusi:** Tambahkan SMOTE atau class weights dalam training

5. [CONFIRMED] Hyperparameter Tidak Spesifik

- **Solusi:** Tentukan nilai konkret (lihat Pertanyaan 17)

6. [CONFIRMED] Ketidakjelasan Integrasi Sentimen dan Topik

- **Solusi:** Lakukan topic modeling per kategori sentimen

C. SARAN PENYEMPURNAAN METODOLOGI

Checklist Perbaikan:

No	Item	Status di Proposal	Rekomendasi
1	Periode data	7 hari	Minimal 4 minggu
2	Inter-rater reliability	Tidak ada	Cohen's Kappa ≥ 0.60
3	Hyperparameter	Tidak spesifik	LR=2e-5, Batch=16, Epoch=3
4	Threshold evaluasi	Tidak ada	Accuracy $\geq 80\%$, F1 ≥ 0.75
5	Class imbalance	Tidak ada	SMOTE atau class weights
6	BERTopic params	Tidak ada	min_cluster_size, n_neighbors
7	Sentimen-topik link	Tidak jelas	Topic modeling per sentimen

D. RINGKASAN PRIORITAS PERTANYAAN

Pertanyaan KRITIS (Wajib Ditanyakan):

1. **Pertanyaan 17** — Hyperparameter fine-tuning (harus ada nilai konkret)
2. **Pertanyaan 18** — Threshold evaluasi (harus ada target)
3. **Pertanyaan 12** — Inter-rater reliability (fundamental)
4. **Pertanyaan 3** — Kecukupan data 7 hari
5. **Pertanyaan 16** — Penanganan class imbalance

Pertanyaan Penting:

1. **Pertanyaan 19** — Parameter BERTopic
2. **Pertanyaan 4** — Hubungan sentimen dan topik
3. **Pertanyaan 11** — Etika scraping

Pertanyaan Tambahan:

1. **Pertanyaan 7** — Perbedaan IndoBERT vs IndoBERTweet
2. **Pertanyaan 21** — Kontribusi bidang SI

E. SUMBER VERIFIKASI (WEB SEARCH)

Topik	Sumber
IndoBERTweet	ACL Anthology
BERTopic	Official Docs
BERT Fine-tuning	Stanford CS224N
Cohen's Kappa	PMC
Class Imbalance	Springer
Twitter Scraping Ethics	arXiv
Roblox Statistics	Statista